An Empirical Evaluation of In-Memory Multi-Version Concurrency Control

Yingjun Wu
National University of Singapore
yingjun@comp.nus.edu.sg

Joy Arulraj
Carnegie Mellon University
jarulraj@cs.cmu.edu

Jiexi Lin
Carnegie Mellon University
jiexil@cs.cmu.edu

Ran Xian
Carnegie Mellon University
rxian@cs.cmu.edu

Andrew Pavlo
Carnegie Mellon University
pavlo@cs.cmu.edu

ABSTRACT

Multi-version concurrency control (MVCC) is currently the most popular transaction management scheme in modern database management systems (DBMSs). Although MVCC was discovered in the late 1970s, it is used in almost every major relational DBMS released in the last decade. Maintaining multiple versions of data potentially increases parallelism without sacrificing serializability when processing transactions. But scaling MVCC in a multi-core and in-memory setting is non-trivial: when there are a large number of threads running in parallel, the synchronization overhead can outweigh the benefits of multi-versioning.

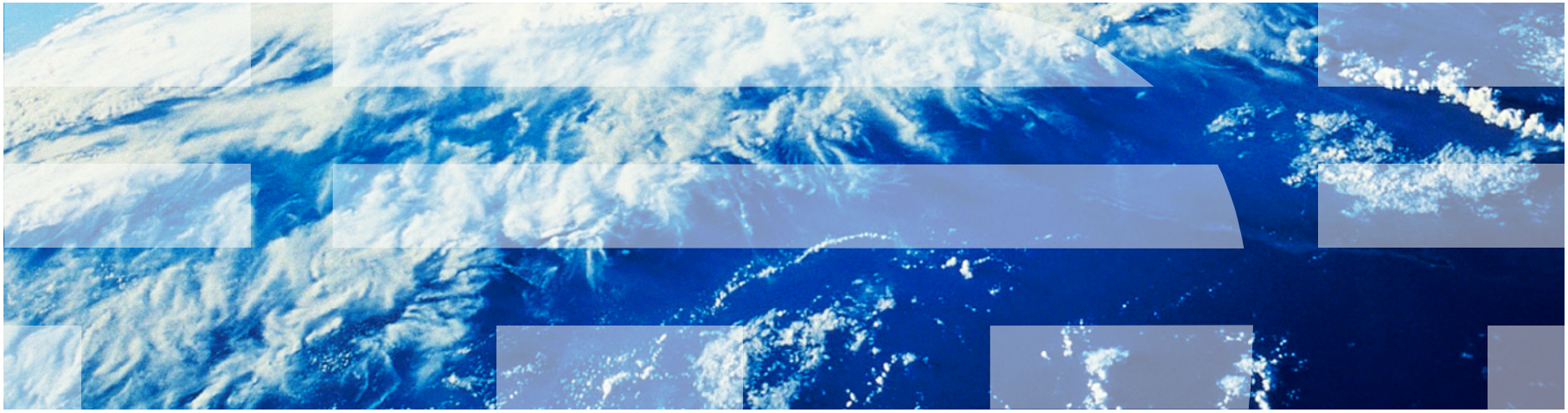
To understand how MVCC perform when processing transactions in modern hardware settings, we conduct an extensive study of the scheme's four key design decisions: concurrency control protocol, version storage, garbage collection, and index management. We implemented state-of-the-art variants of all of these in an in-memory DBMS and evaluated them using OLTP workloads. Our analysis identifies the fundamental bottlenecks of each design choice.

in a 1979 dissertation [38] and the first implementation started in 1981 [22] for the InterBase DBMS (now open-sourced as Firebird). MVCC is also used in some of the most widely deployed disk-oriented DBMSs today, including Oracle (since 1984 [4]), Postgres (since 1985 [41]), and MySQL's InnoDB engine (since 2001). But while there are plenty of contemporaries to these older systems that use a single-version scheme (e.g., IBM DB2, Sybase), almost every new transactional DBMS eschews this approach in favor of MVCC [37]. This includes both commercial (e.g., Microsoft Hekaton [16], SAP HANA [40], MemSQL [1], NuoDB [3]) and academic (e.g., HYRISE [21], HyPer [36]) systems.

Despite all these newer systems using MVCC, there is no one "standard" implementation. There are several design choices that have different trade-offs and performance behaviors. Until now, there has not been a comprehensive evaluation of MVCC in a modern DBMS operating environment. The last extensive study was in the 1980s [13], but it used simulated workloads running in a disk-oriented DBMS with a single CPU core. The design choices of legacy disk-oriented DBMSs are inappropriate for in-memory

https://www.vldb.org/pvldb/vol10/p781-Wu.pdf

Indexing



Indexing

- Indexing mechanisms used to speed up access to desired data.
  - E.g., author catalog in library
- **Search Key**: attribute used to look up records in a file.
- An **index** consists of records (called **index entries**) of the form

search-key	pointer

- The index is typically much smaller than the original data
  - E.g., 100X smaller
- How big does the pointer need to be?
  - Example: A pointer to a specific byte within 1GB of data
    - 1GB can be represented as an array with 30 bits (0's or 1's)
    - 000…00 points to the start of 1GB
    - 000…01 points to the first byte
  - General idea: take the size of the data, and figure out how many bits need to represent it
- Where would you store the index?
  - Usually in memory
  - → Index has to be small, since memory capacity is limited

Index Evaluation Metrics

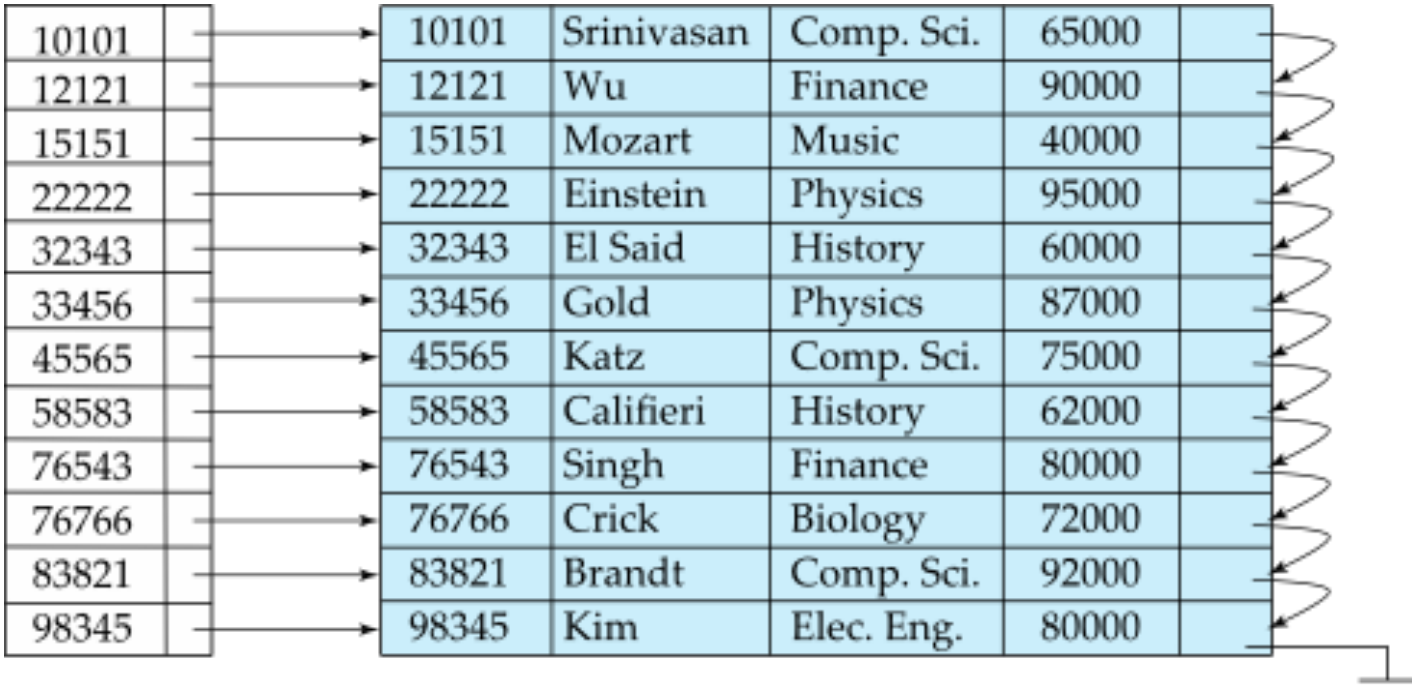
- Access types supported efficiently:
  - Records with a specified value in the attribute
  - Records with an attribute value falling in a specified range of values
- Access time
- Insertion time
- Update time
- Deletion time
- Space overhead

Ordered Indices

- In an **ordered index,** index entries sorted on the search key
- **Primary index:** in a sequentially ordered file, the index whose search key specifies the sequential order of the file
  - The search key of a primary index is usually but not necessarily the primary key
- **Secondary index**: an index whose search key specifies an order different from the sequential order of the file
  - We'll show what this means in a bit

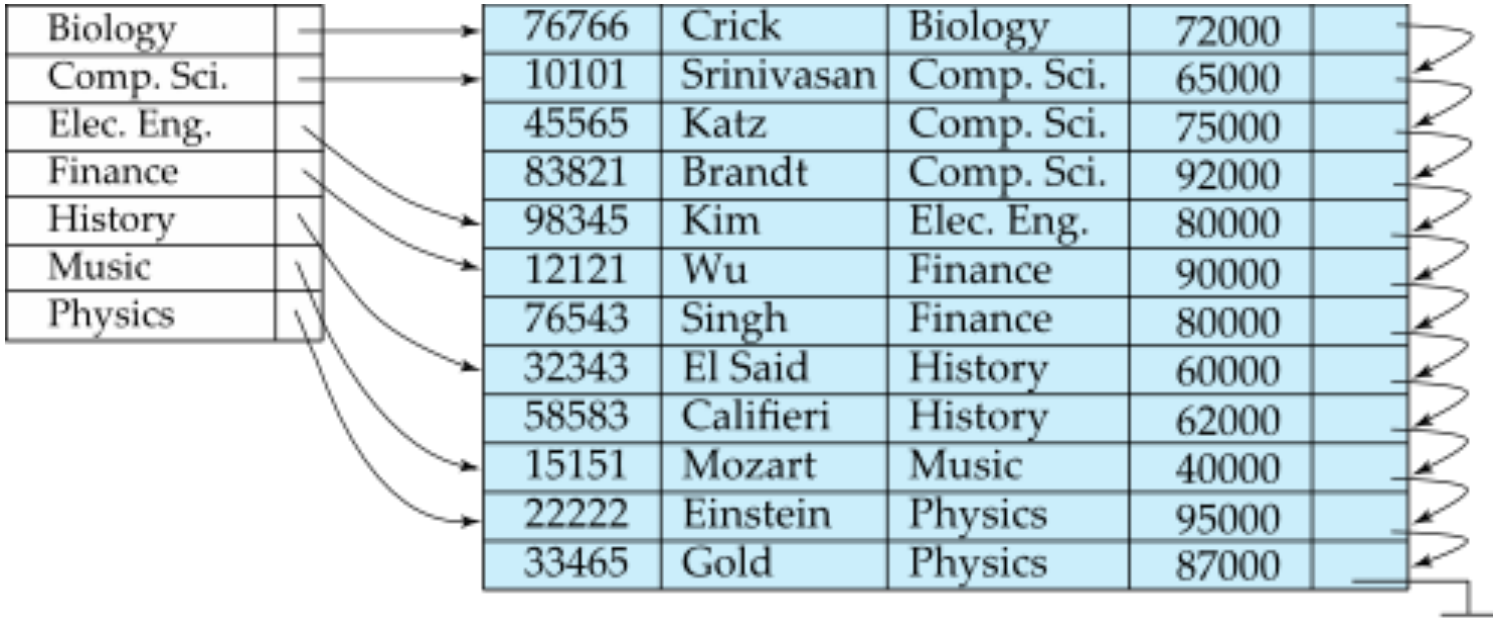
Dense Index Files

- **Dense index** — Index record appears for every search-key value in the database.
- E.g. index on *ID* attribute of *instructor* relation



Dense Index Files: non-unique attribute as search key

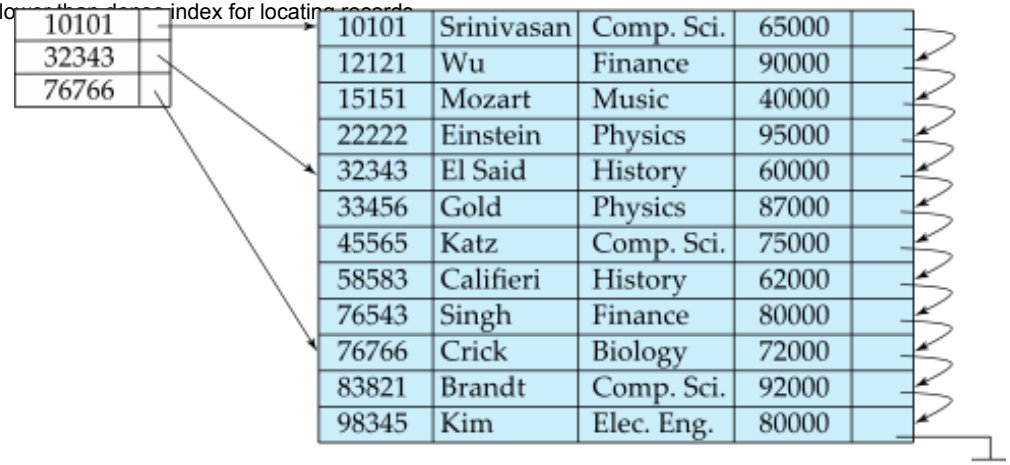
- Dense index on *dept_name*, with *instructor* file sorted on *dept_name*



Sparse Index Files

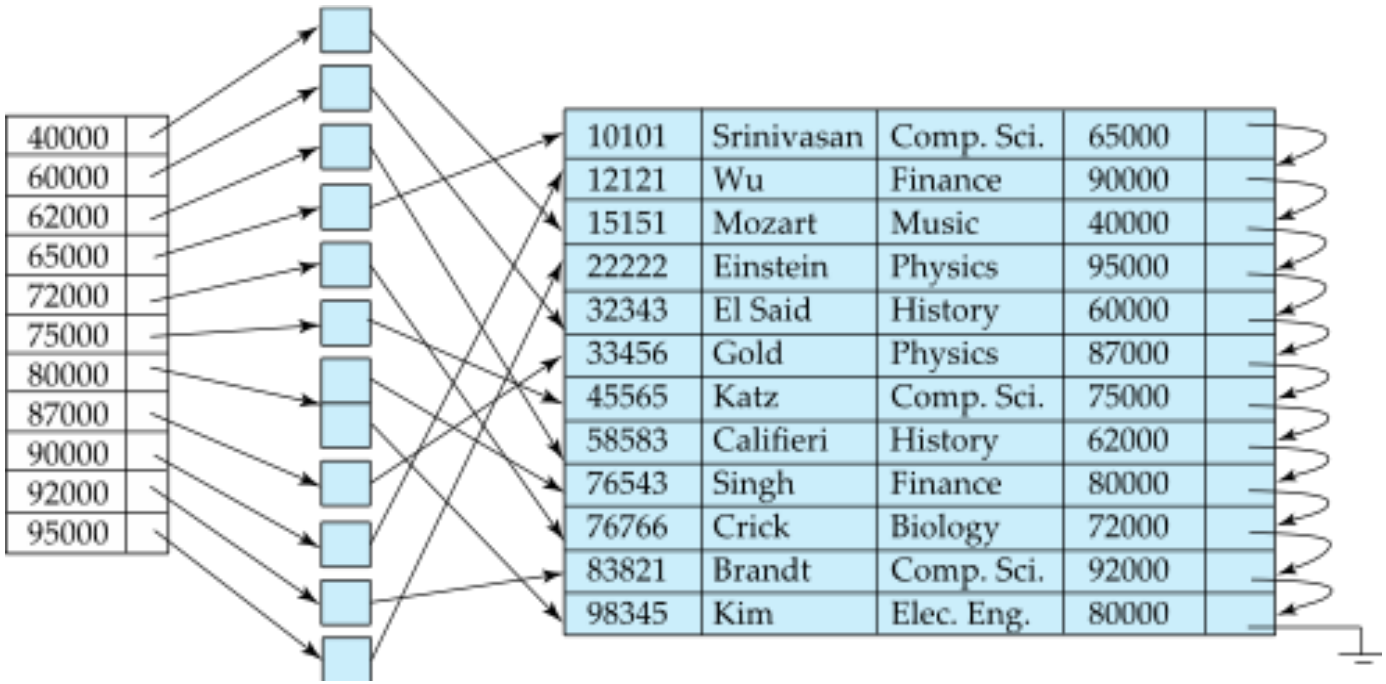
- **Sparse Index**: contains index records for only some search-key values.
  - Applicable when records are sequentially ordered on search-key
- To locate a record with search-key value *K* we:
  - Find index record with largest search-key value < *K*
  - Search file sequentially starting at the record to which the index record points

- Compared to dense indices:
  - Less space and less maintenance overhead for insertions and deletions
  - Generally slower than dense index for locating records



Secondary Index Example

- Secondary index on salary field of instructor

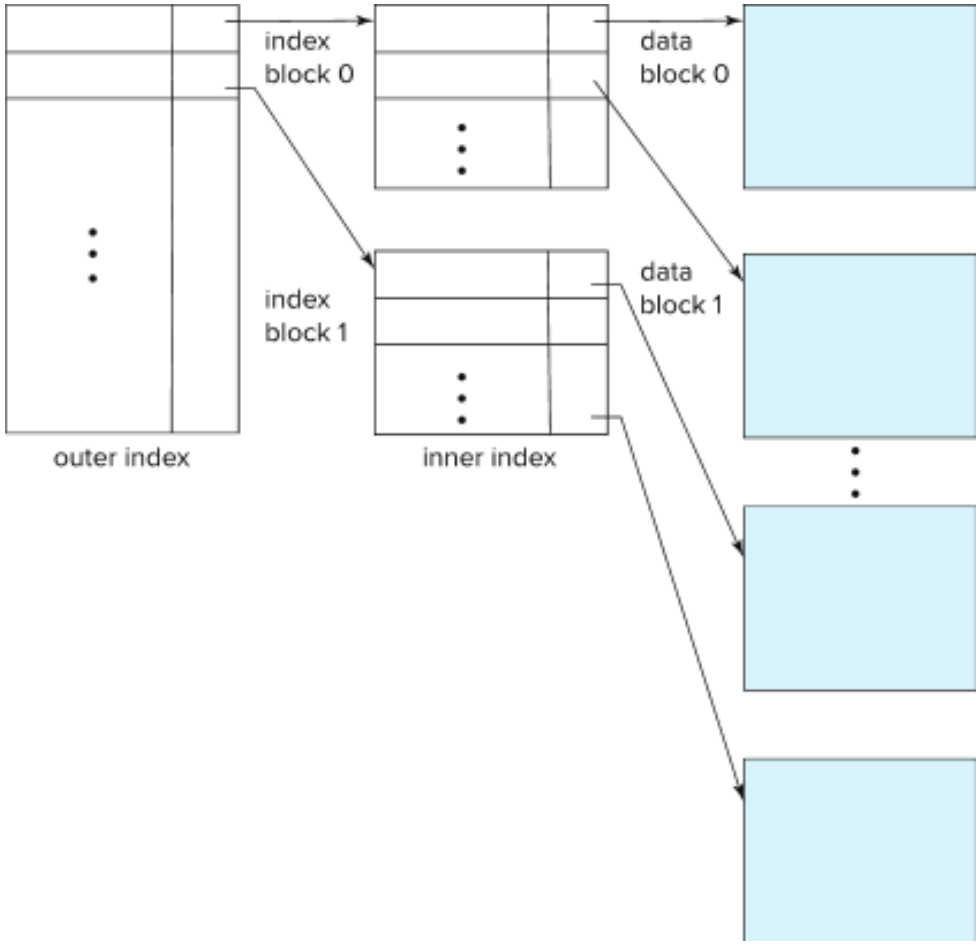


- Index record points to a bucket that contains pointers to all the actual records with that particular search-key value.
- Secondary indices have to be dense
  - Why?

Multilevel Index

- If index does not fit in memory, access becomes expensive
  - Usually databases try to keep the index in memory if they can!
- Solution: treat index kept on disk as a sequential file and construct a sparse index on it
  - outer index – a sparse index of the basic index
  - inner index – the basic index file
- If even outer index is too large to fit in main memory, yet another level of index can be created, and so on.
- Indices at all levels must be updated on insertion or deletion from the file.
- File systems often use a multilevel index (e.g., nested directories)

Multilevel Index (continued)



Index Update: Insertion

- **Single-level index insertion:**
  - Perform a lookup using the search-key value of the record to be inserted.
  - **Dense indices** – if the search-key value does not appear in the index, insert it
    - Indices are maintained as sequential files
    - Need to create space for new entry, overflow blocks may be required
  - **Sparse indices** – if index stores an entry for each block of the file, no change needs to be made to the index unless a new block is created
    - If a new block is created, the first search-key value appearing in the new block is inserted into the index
- **Multilevel insertion and deletion:** algorithms are simple extensions of the single-level algorithms